

Demand-Paged Virtual Memory and Page Caching

Main Points

- Demand-paged Virtual Memory
- Cache Replacement Policies
 - FIFO, MIN, LRU, LFU, Clock
- Memory-mapped files
- Use cases

Virtual Memory

- The OS provides applications the abstraction of more memory than is physically present
- The abstraction is implemented considering the main memory *as a cache for disk(s)*
- Application pages that fits in memory have valid page table entries
- Pages not resident in memory have invalid page table entries and are loaded *on-demand* upon access by the OS
 - Such process is called *demand paging*

Demand Paging

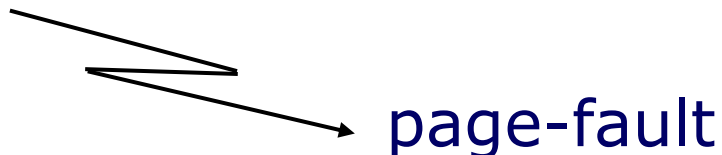
Physical page field	flags				
Number of physical page if P=1; address on disk of the page if P=0	R	W	U	M	P

Page descriptor

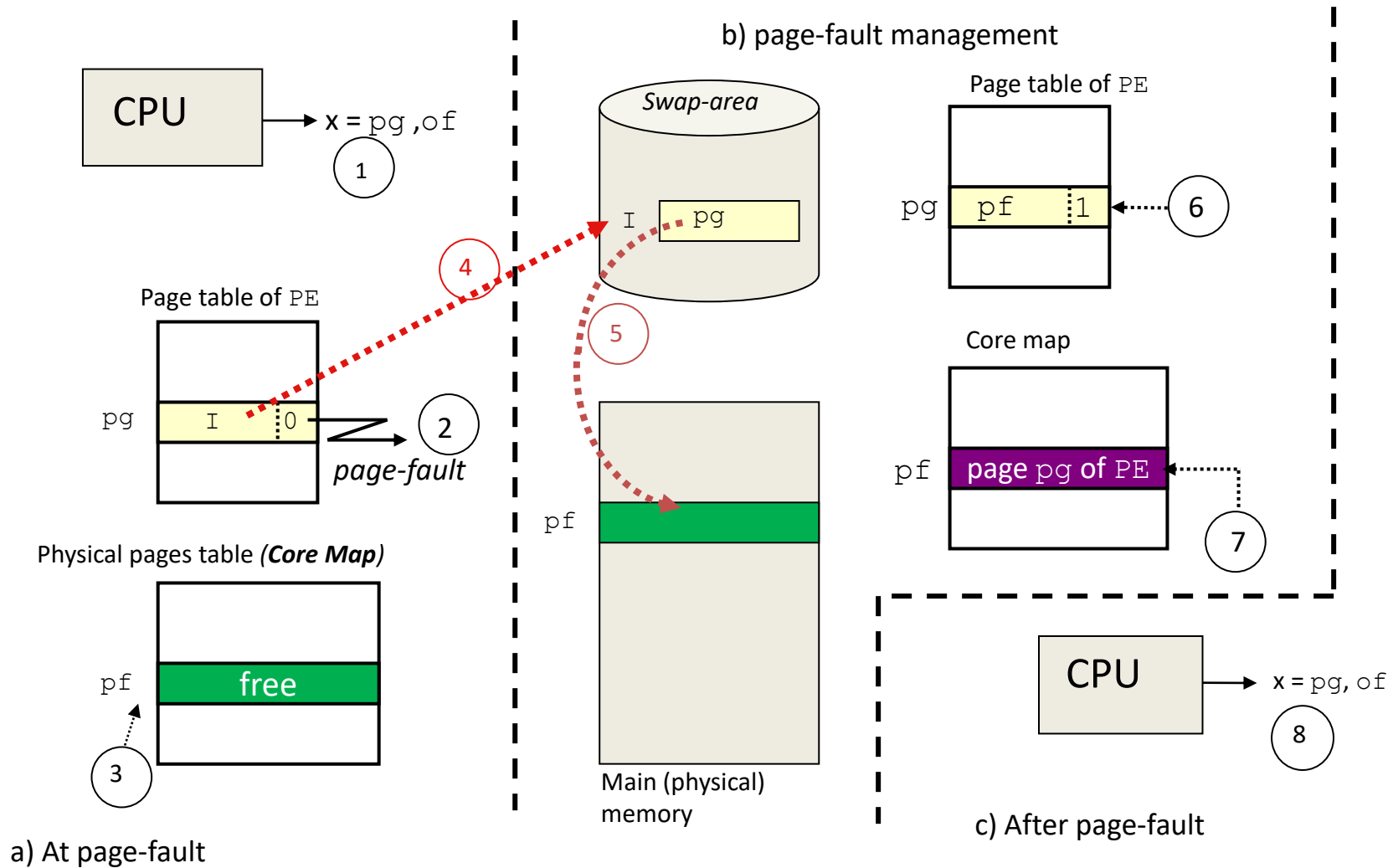
The page table contains a page descriptor for each page
Beyond the information for translation of the address, the descriptor contains some flags:

- R, W: read/write access rights
- M, U: modified/use bits (for the page replacement algorithms)
- **P: presence bit (page is invalid if not present in main memory)**

- P = 1: page in main memory
- P = 0: page not in main memory



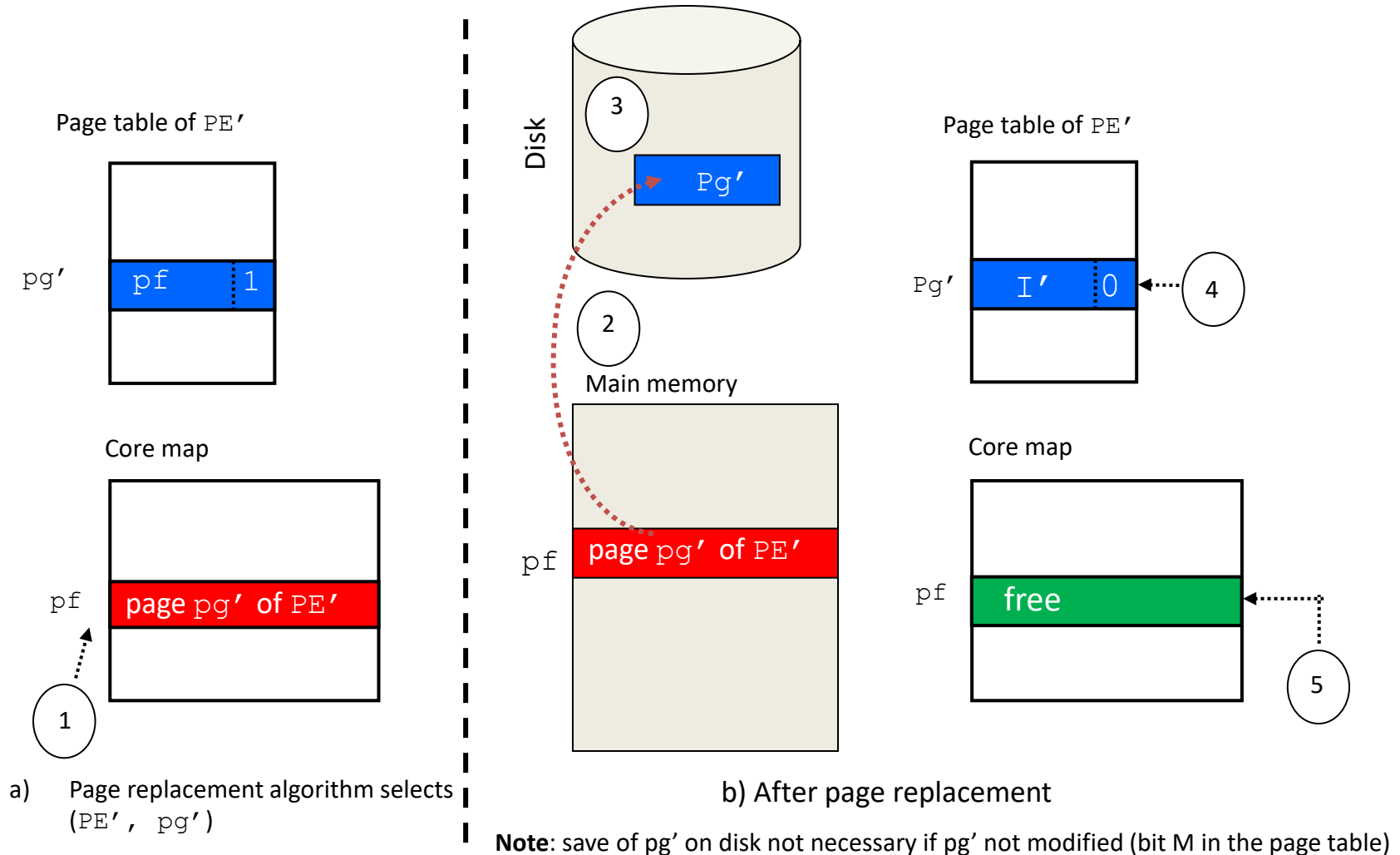
Page fault management



On-demand Paging

1. TLB miss
2. Page table walk
3. Page fault (page invalid in page table)
4. Trap to kernel
5. Convert address to file + offset
6. Allocate page frame
 - Evict page if needed
7. Initiate disk block read into page frame
8. Disk interrupt when DMA complete
9. Mark page as valid
10. Resume process at faulting instruction
11. TLB miss
12. Page table walk to fetch translation
13. Execute instruction

Page replacement



Allocating a Page Frame

- Select old page to evict
- Find all page table entries that refer to old page
 - If page frame is shared
- Set each page table entry to invalid
- Remove any TLB entries
 - Copies of now invalid page table entry
- Write changes to page to disk, if necessary
 - i.e. if the page had been modified

How do we know if page has been modified?

- Every page table entry has some bookkeeping
 - Has page been modified?
 - Set by hardware on *store* instruction to page
 - In both TLB and page table entry
 - Has page been used?
 - Set by hardware on *load* or *store* instruction to page
 - In page table entry on a TLB miss
- Can be reset by the OS kernel
 - When changes to page are flushed to disk
 - To track whether page is recently used

TLB

	dirty = 0

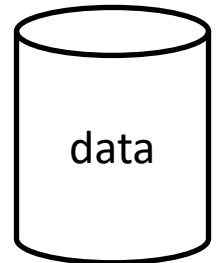
Page Table

Frame	Access
	dirty = 0

Physical memory
page frames

data

disk



TLB

	dirty = 1

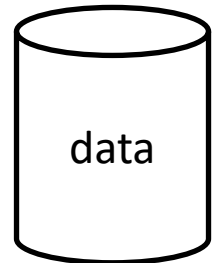
Page Table

Frame	Access
	dirty = 1

Physical memory
page frames

new data

disk



Emulating M and U bits

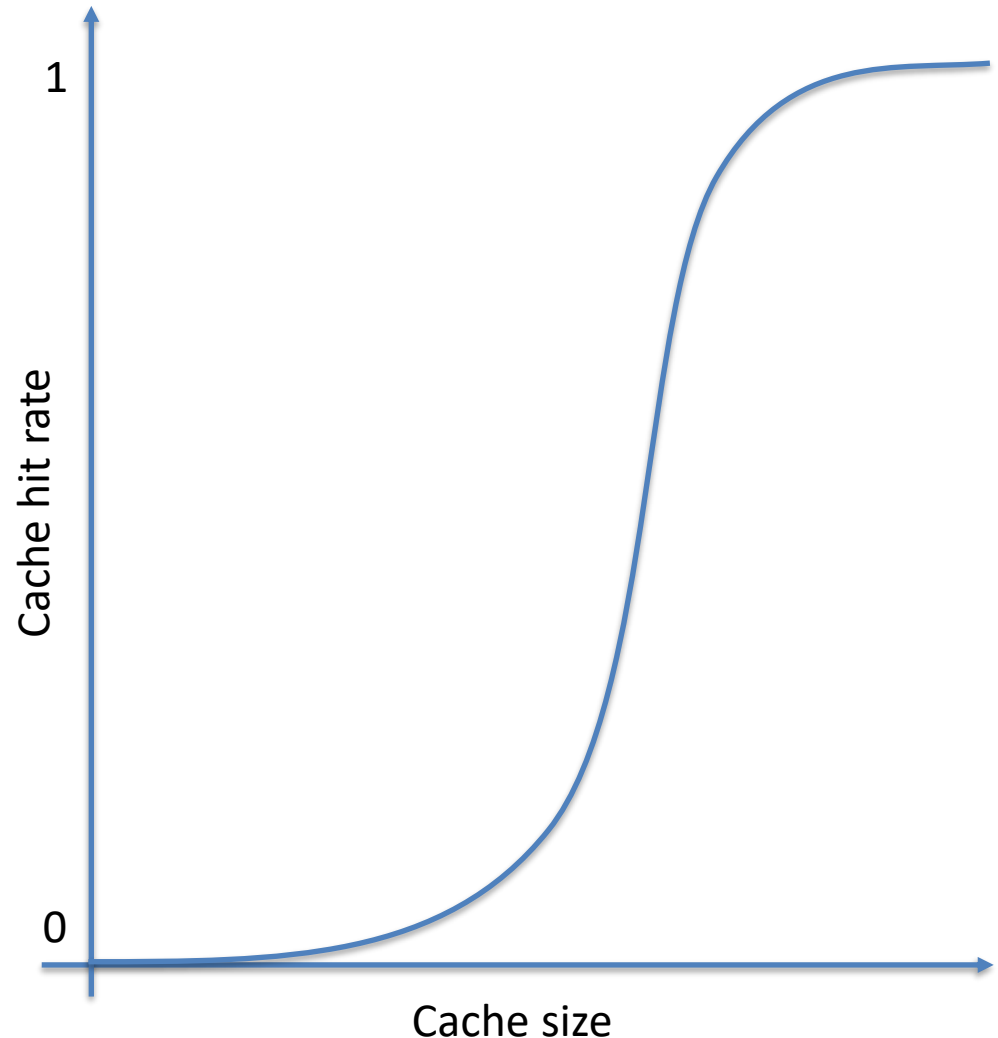
- Some processor architectures do not keep a use bit in the page table entry
 - Extra bookkeeping and complexity
- OS can emulate a use bit:
 - For M bit: set all clean pages as read-only
 - For U bit: set all unused pages as invalid
 - On first write (M) or read/write (U), take page fault to kernel
 - From info in the **core map** the kernel knows what to do
 - Kernel sets M bit and marks page as read-write
 - Kernel sets U bit and marks page as read or read/write

Demand-paged Virtual Memory

- Problem:
 - If the OS pages out to disk (swap) a page just before it is about to be used, then it will have to get that page again almost immediately
 - Too much of this leads to a condition known as *thrashing*
 - Thrashing: the system spends most of the time swapping pages to disk rather than executing application's instructions

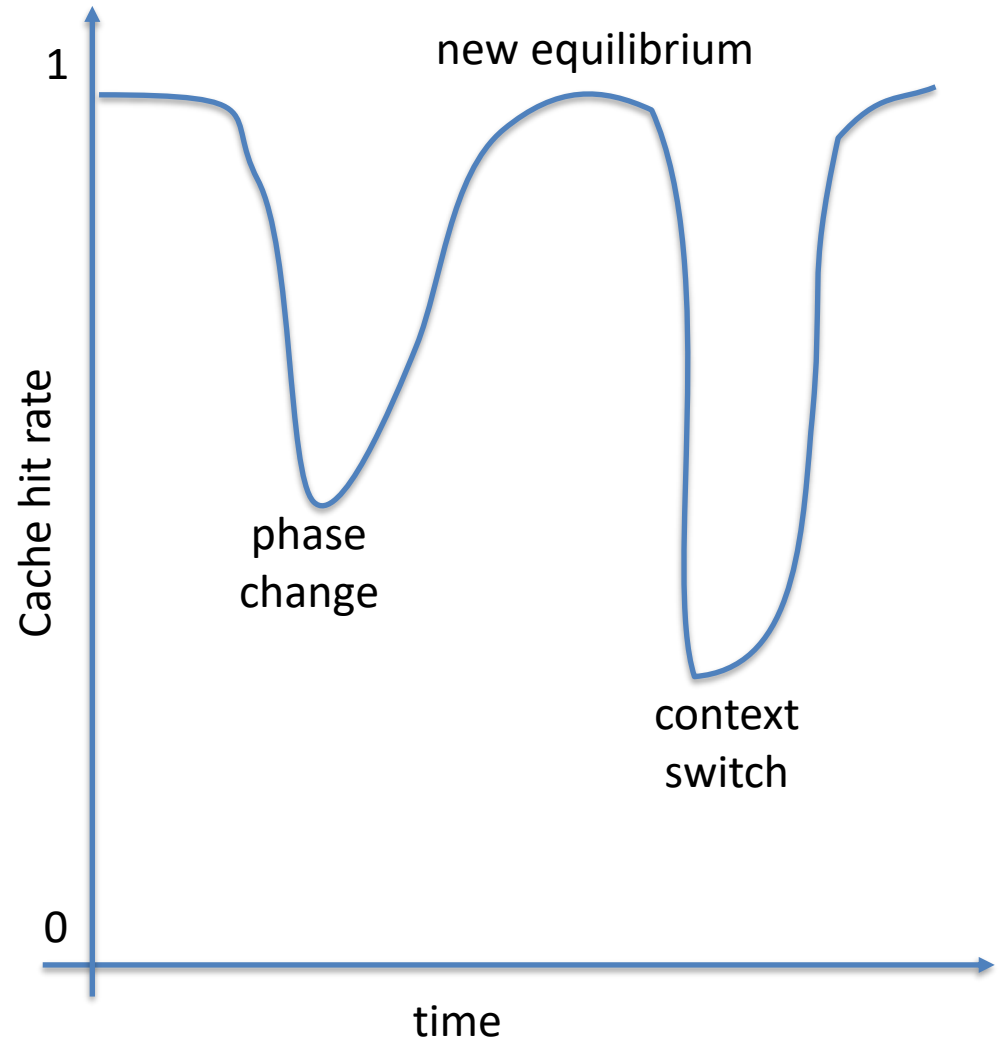
Working Set

- Working Set: set of memory locations that need to be cached for reasonable cache hit rate
- Thrashing happens when system has a too small cache



Phase Change Behavior

- Programs may change their working set over time
- Context switches also change working set



Cache Replacement Policy

- On a cache miss, how do we choose which entry to replace?
 - Assuming the new entry is more likely to be used soon
- Policy goal: *reduce cache misses*
 - Improve expected case performance
 - Also: reduce likelihood of very poor performance

A Simple Policy

- Random?
 - Replace a random page
- FIFO?
 - Replace the page that has been in the cache (main memory) the longest time
 - What could go wrong?

FIFO in Action

reference	A	B	C	D	E	A	B	C	D	E	A	B	C	D	E
1	A				E				D				C		
2		B				A				E				D	
3			C				B				A				E
4				D				C				B			

Worst case for FIFO is when program strides through memory that is larger than the main memory

- working set larger than cache capacity

MIN, LRU, NRU

- **MIN** (ideal, optimal)
 - Replace the page that will not be used for the longest time into the future (i.e., whose ***future distance*** is maximum)
 - Optimality proof based on exchange: if evict a page used sooner, that will trigger an earlier page fault (=cache miss)
- Least Recently Used (**LRU**)
 - Replace the page that has not been used for the longest time in the past (i.e., whose ***past distance*** is maximum)
 - Approximation of MIN
- Not Recently used (**NRU**)
 - Replace one of the pages that have not been used recently
 - Relax the requirement of LRU
 - Approximation of LRU, easier to implement
 - LFU (Least Frequently Used) replace the cache entry used the least often
 - Examples: second chance, working set algorithms

LRU/MIN for Sequential Scan

LRU

reference	A	B	C	D	E	A	B	C	D	E	A	B	C	D	E
1	A				E				D				C		
2		B				A				E				D	
3			C				B				A				E
4				D				C				B			

MIN

reference	A	B	C	D	E	A	B	C	D	E	A	B	C	D	E
1	A					+					+				
2		B					+					+	C		
3			C					+	D					+	
4				D	E					+					+

LRU

reference	A	B	A	C	B	D	A	D	E	D	A	E	B	A	C
1	A		+				+				+			+	
2		B			+								+		
3				C					E			+			
4						D		+		+					C

FIFO

reference	A	B	A	C	B	D	A	D	E	D	A	E	B	A	C
1	A		+				+		E			+			
2		B			+						A			+	
3				C									B		
4						D		+		+					C

MIN

reference	A	B	A	C	B	D	A	D	E	D	A	E	B	A	C
1	A		+				+				+			+	
2		B			+								+		
3				C					E			+			
4						D		+		+					C

Belady's Anomaly

FIFO (3 slots)

reference	A	B	C	D	A	B	E	A	B	C	D	E
1	A			D			E					+
2		B			A			+		C		
3			C			B			+		D	

FIFO (4 slots)

reference	A	B	C	D	A	B	E	A	B	C	D	E
1	A				+		E				D	
2		B				+		A				E
3			C						B			
4				D						C		

Estimating LRU: Second Chance

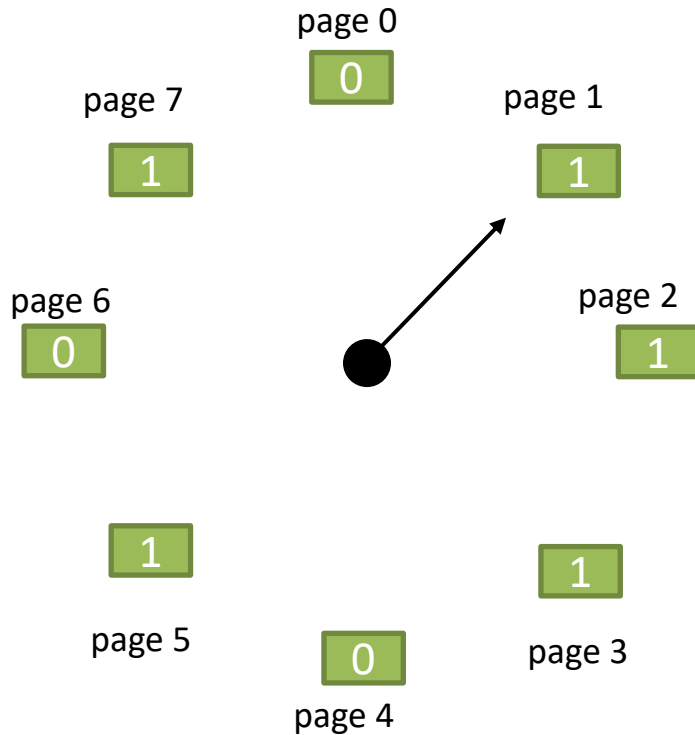
- **Second Chance:**
 - FIFO algorithm with a reference bit (R)
 - The page-frames are ordered in a linked list
 - If a page is needed check the oldest (head)
 - If R is zero take it as victim
 - Otherwise, R=0, and move it at the end of the list (giving it a second chance)
 - Keep going on until a 0 referenced bit page is found

PROBLEM: too many page frame move in the list

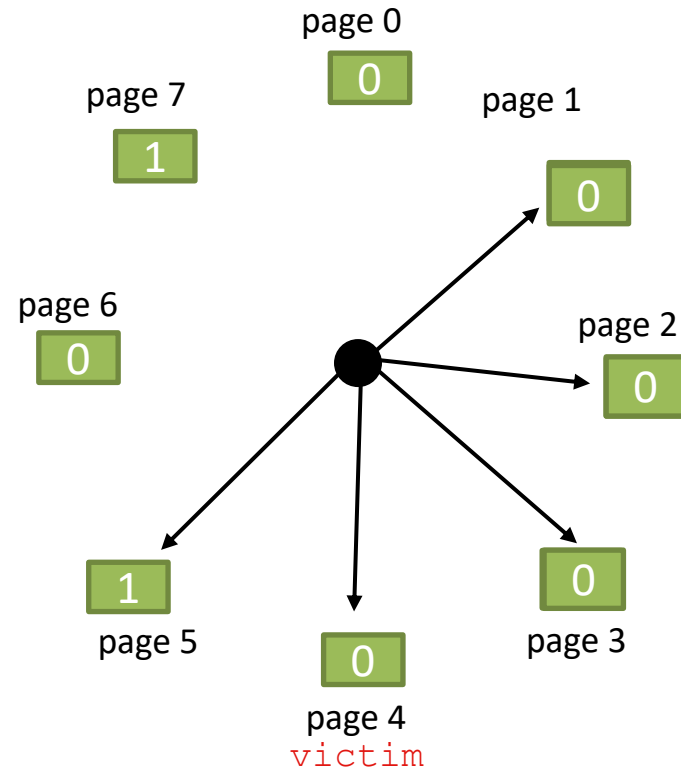
Estimating LRU: Clock Algorithm

- Basically, Second Chance implemented with a circular list
- Periodically, sweep through all pages
- If page is unused (R is 0), reclaim
- If page is used, mark as unused ($R=0$) and advance to the next

Clock algorithm example



a) Initial state



b) Next states of second chance

Nth Chance: generalization of Second Chance

- Instead of one bit per page, keep an integer
 - notInUseSince: # of sweeps since last use
- Periodically, sweep through all page frames
- If page hasn't been used in any of the past N sweeps, reclaim
- If page is used, mark as unused and set as active in current sweep

```
if (page is used) {  
    notInUseSince = 0;  
} else if (notInUseSince < N) {  
    notInUseSince++;  
} else {  
    reclaim page;  
}
```

Local and global page replacement

- Global algorithms:
 - The page considered for removal is selected among all pages in main memory
 - Irrespective of the owner
 - “past distance” of a pages defined based on a global time (absolute clock)
 - May result in trashing of slow processes
- Local algorithms
 - The page selected for removal belongs to the process that caused the page fault
 - Fair with “slow” processes about trashing
 - Past distance of a page based on relative time
 - The time a process has spent in running state

Local vs global page replacement

T: time of last reference

a)

	T
A0	10
A1	7
A2	5
B0	9
B1	6
C0	12
C1	4
C2	3

b)

	T
A0	10
A1	7
A2	5
B0	9
B1	6
C0	12
C1	4
C2	3

c)

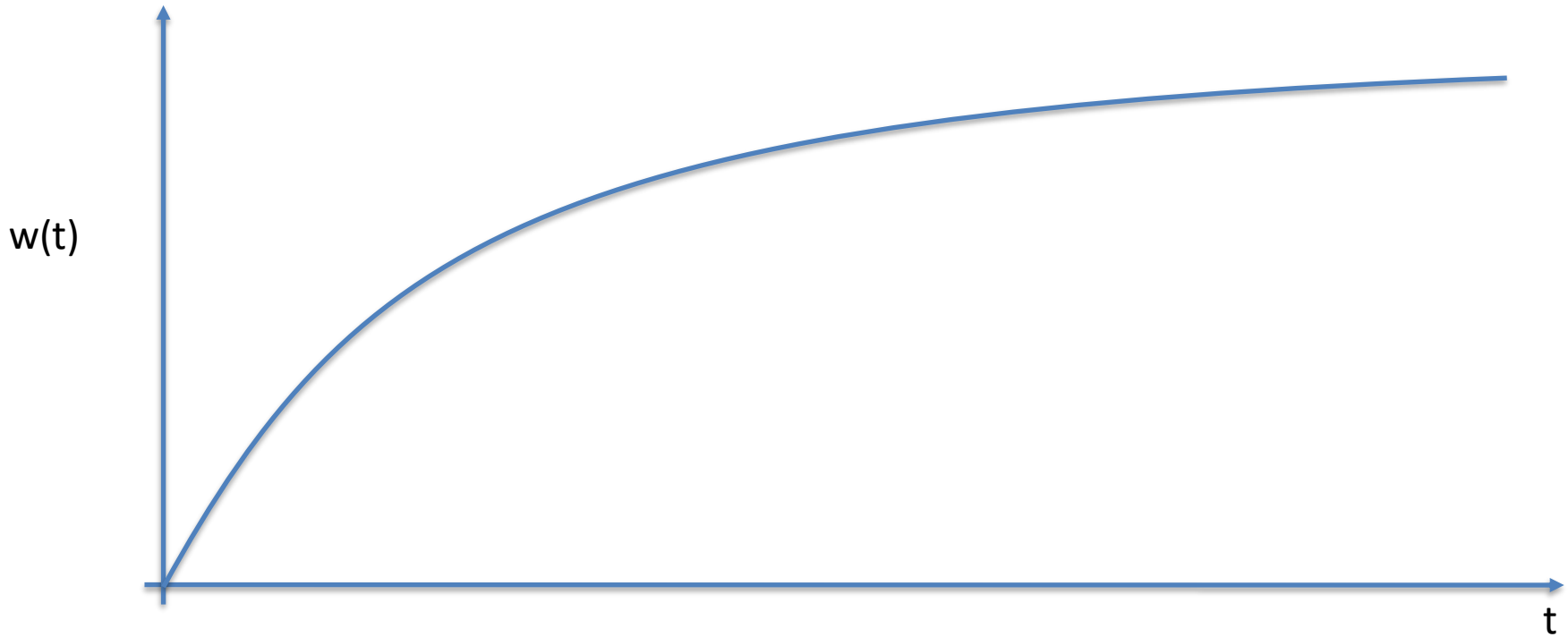
	T
A0	10
A1	7
A2	5
B0	9
B1	6
C0	12
C1	4
C2	3

- a) Initial configuration
- b) Page replacement with a local policy (WS, LRU, sec. chance)
- c) Page replacement with a global policy (LRU, sec. chance)

Working set algorithm

- Keep in memory the *working set of a process*:
 - the pages that the process is currently using
- Working set defined as:
 - *The set of pages referred in the last k memory accesses*
 - difficult and costly to implement
 - *The set of pages referred in the last period T*
 - usually implemented in this way, using the «use» bit

Working set



- Working Set: **The set of pages referred in the last period T**
- $w(t)$ is the size of the working set as function of time

Working set algorithm

- Each process has a number of physical pages reserved to upload its working set
 - WS replacement policy is inherently local
- Resident set:
 - is the actual set of virtual pages in main memory
 - some of them may be out of the working set
 - Resident set $\neq$ working set

Working set algorithm

- WS defined as the set of pages referred in the last period T
 - T is a parameter of the WS algorithm
- For each page:
 - *bit R* (called “referred” or “use” bit) indicates whether the page had been referred in the last time tick
 - *TLR* an approximation of the time of last reference to the page
 - At the end of each time tick resets bit R for each page and updates the approximation of time of last reference
 - *age* of a page defined as the difference between current time and time of last reference

Working set algorithm

When it is executed (either at page fault or periodically by daemon):

- For each page considers its values of R , TLR and age
 - If $R=0$: $age = current_time - TLR$
 - if $R=1$: $TLR = current_time$ ($age = 0$) and resets R
- The pages with $age < T$ (referred in the last period T) are in the working set and (if possible) are not removed

Working set algorithm

Current virtual time: 2204

Page table

age	R	...
2084	1	
2003	0	
1980	1	
1213	0	
2014	1	
2020	1	
1604	0	

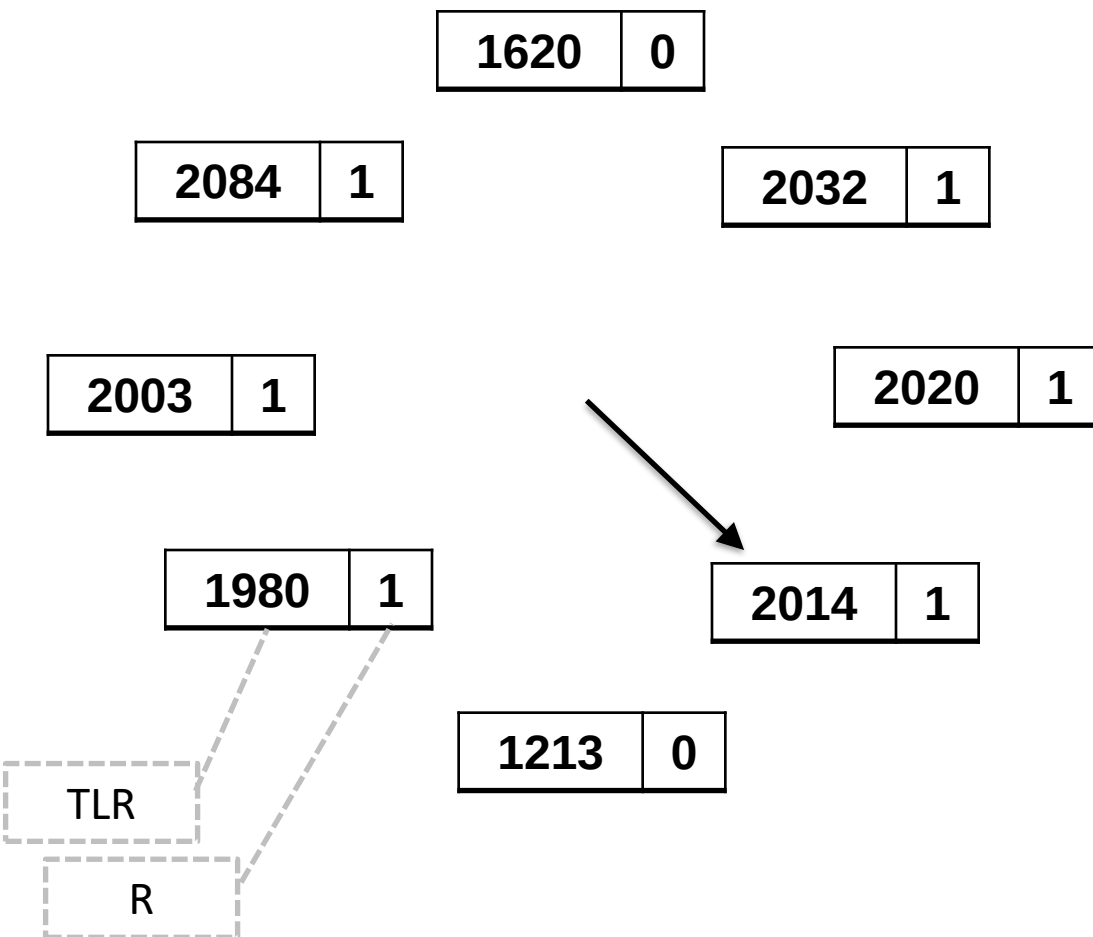
```
For each page: {  
  if (R==0)  
    age = current_time - TLR;  
  else if (R==1) {  
    TLR= current_time; R=0; age=0;  
  }  
  if ( (age>T)  
    removes the page  
}
```

if (age<=T for each page)
 removes the page with higher age

[age: current_time - TLR]

WSClock (working set clock)

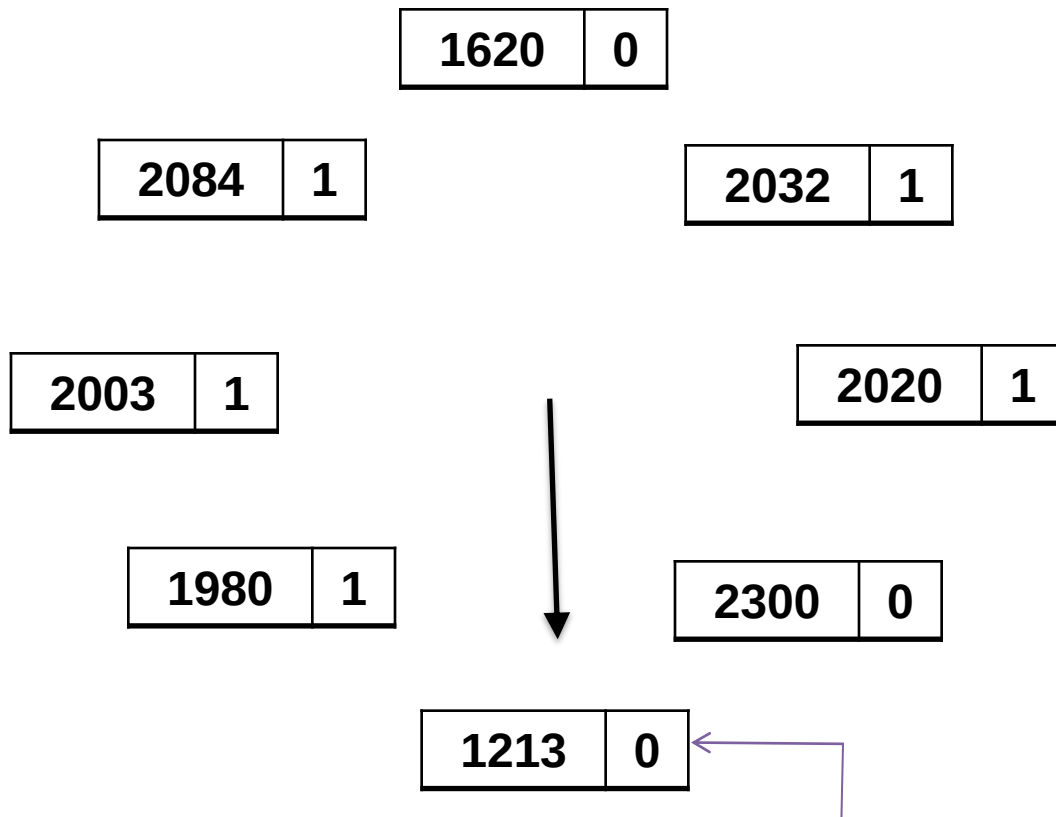
current_time = 2300; T=1000



- **Considers only the pages in main memory**
 - More efficient than scanning the page table
- Pages in a circular list
- At page fault looks for a page out of the WS
 - Better if not “dirty”
 - If selects a dirty page, the page is saved before its actual removal

WSClock (working set clock)

current_time = 2300; T=1000

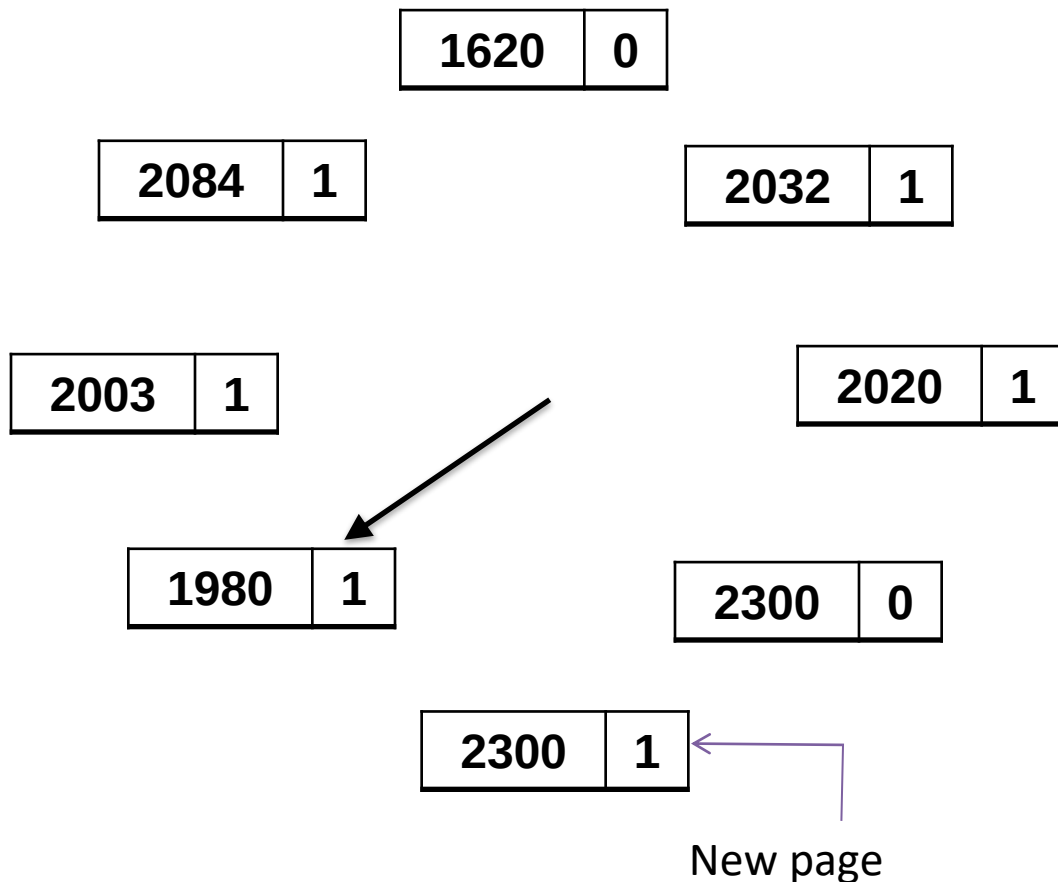


age > T : removes this page

- **Considers only the pages in main memory**
 - More efficient than scanning the page table
- Pages in a circular list
- At page fault looks for a page out of the WS
 - Better if not “dirty”
 - If selects a dirty page, the page is saved before its actual removal

WSClock (working set clock)

current_time = 2300; T=1000



- Considers only the pages in main memory
 - More efficient than scanning the page table
- Pages in a circular list
- At page fault looks for a page out of the WS
 - Better if not “dirty”
 - If selects a dirty page, the page is saved before its actual removal

Working set algorithm

- In practice, WS and all page replacement algorithms are executed in advance
- Guarantees free physical pages in case of page fault
 - To speed up the page fault
- Details in the case studies (Unix & Windows)

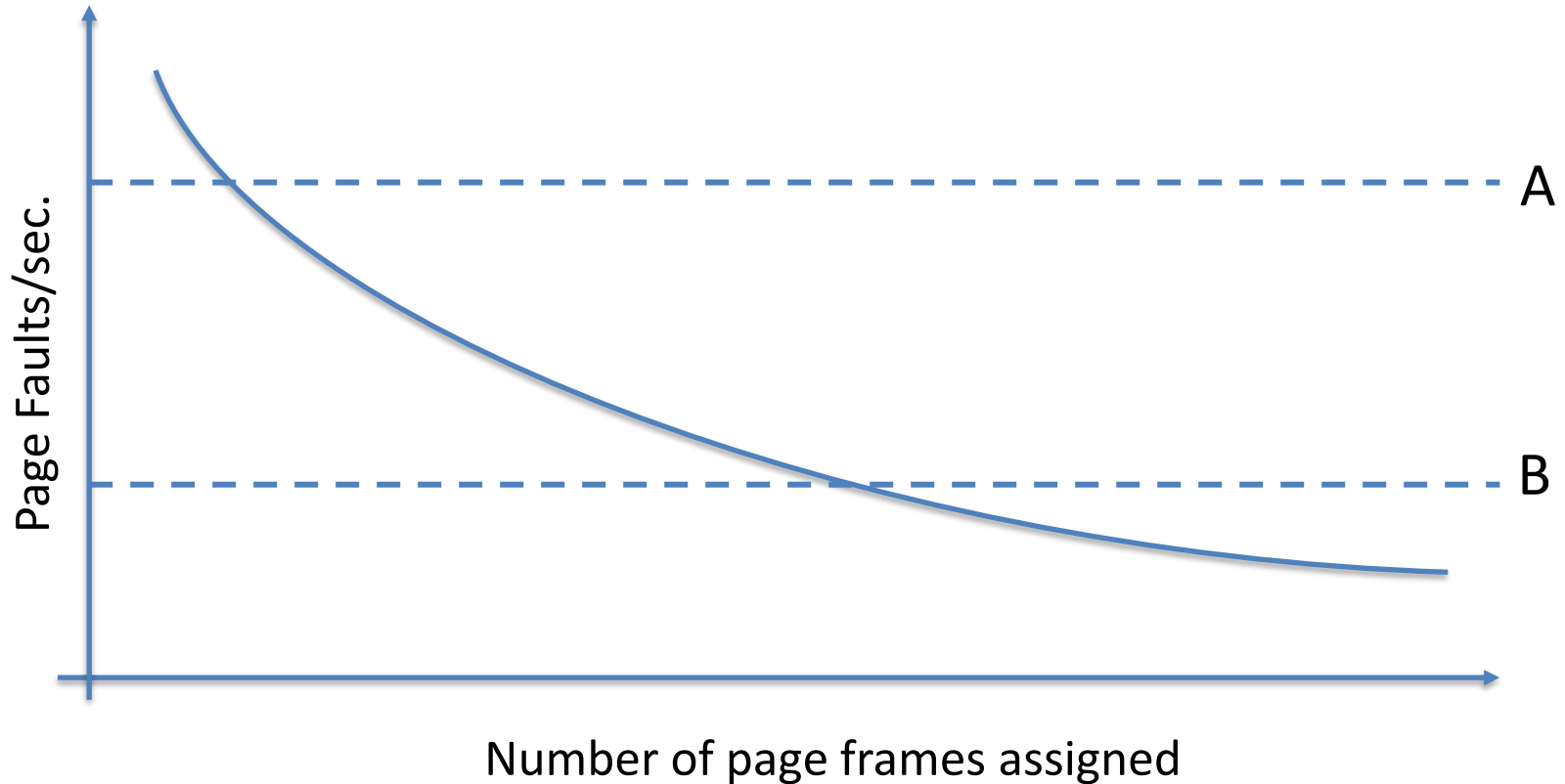
Paging and WS algorithm

- On demand paging:
 - Initially no page of the process is loaded in memory
 - Pages loaded by the process by generating page faults
 - Initially the number of page faults is high
 - When the working set had been loaded the number of page faults reduces
- Prepaging
 - A new process becomes ready when all its pages in the working set are loaded in main memory
 - Need to know (or to predict) what pages will be in the working set initially
 - not easy, can be done for some pages

Working set algorithm

- What should be the number of physical pages allocated to a process?
 - i.e. what should be the (maximum) size of the resident set of a process?
- Not easy to say in advance
- Page allocation algorithms
 - Static algorithms do not work
 - Dynamic algorithms: *Page Fault Frequency* (PFF)

Page Fault Frequency



Number of page faults per unit of time, depending on the number of physical pages assigned to the process

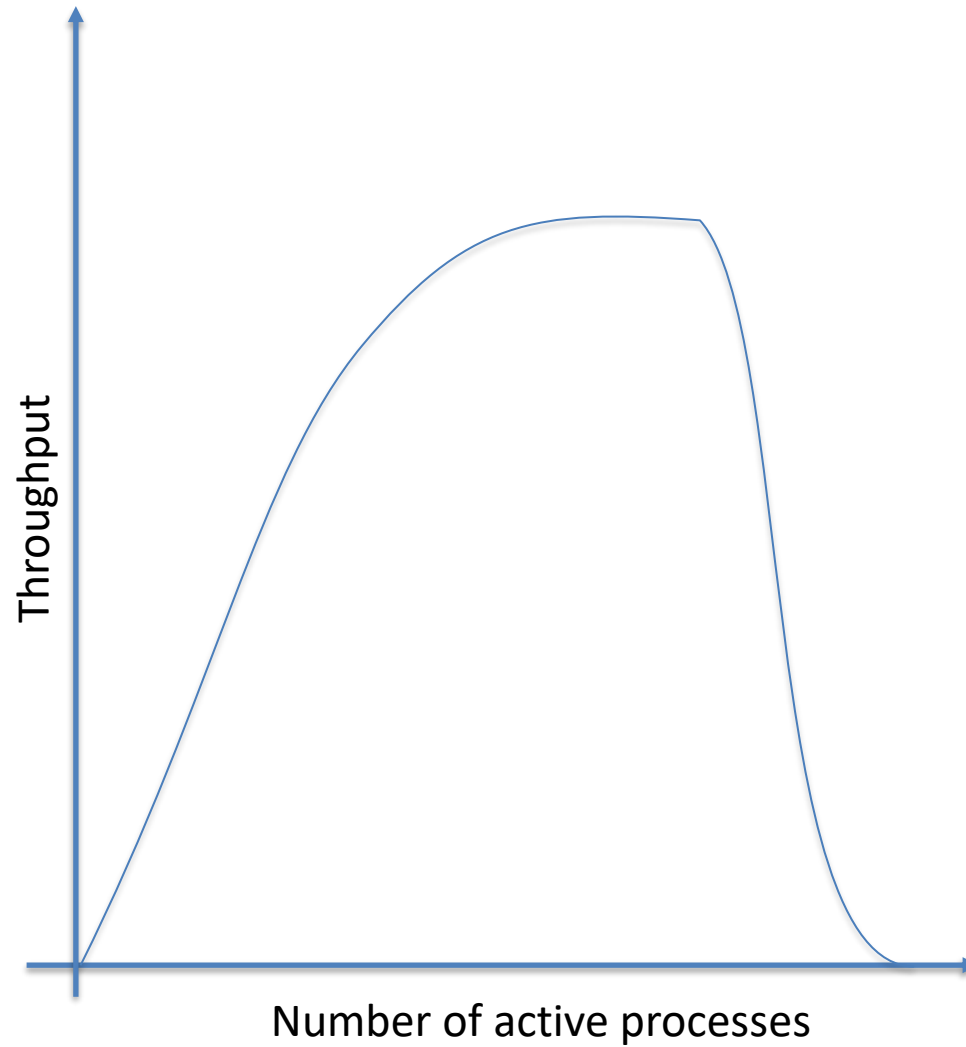
Page Fault Frequency

- Dynamically determines the number of physical pages assigned to a process
 - *Guarantees that resident set $\geq$ working set*
- When frequency of page faults $\gg$ «natural frequency»
 - Increases the size of the resident set
- When frequency of page faults $\ll$ «natural frequency»
 - Reduces the size of the resident set

Question

- What happens to system performance as we increase the number of processes?
 - If the sum of the working sets becomes bigger than the physical memory?

Thrashing



Thrashing

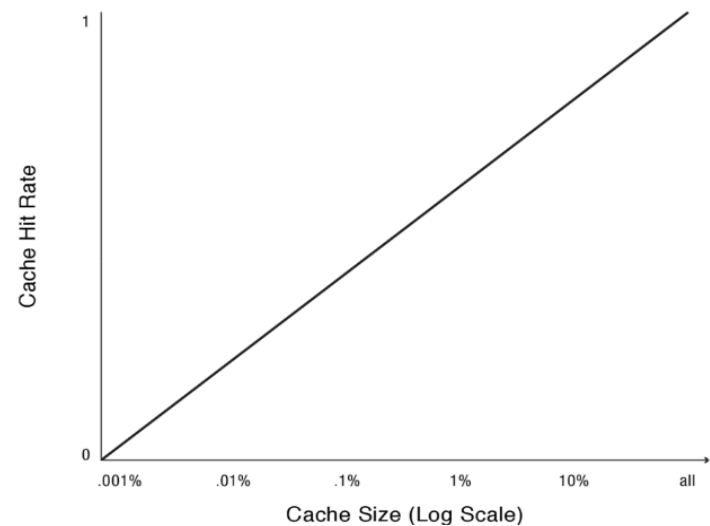
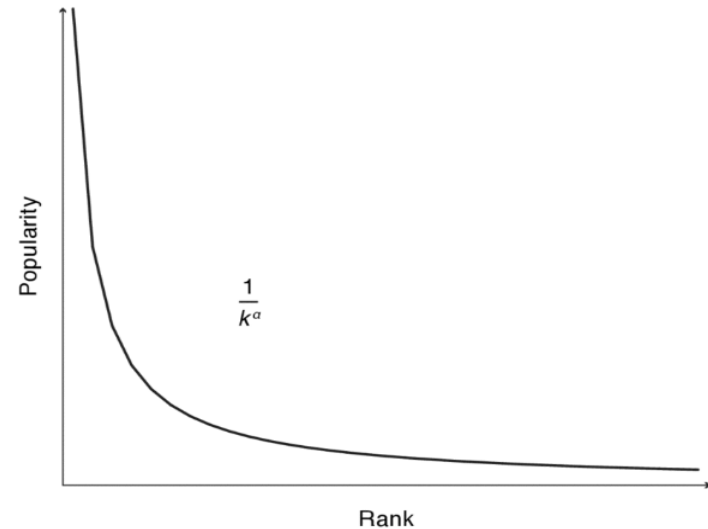
- When, from the PFF algorithm, comes out:
 - Some processes require more memory
 - No process requires less memory
- The number of page faults raises up
 - The system almost halts...
- Solution: reduce the number of processes in main memory
 - Reduces competition for memory (reduces the **degree of multiprogramming**)
 - Swaps out some process on disk

Where pages are stored

- Every process segment backed by a file on disk
 - Code segment -> code portion of executable
 - Data, heap, stack segments -> temp files
 - Shared libraries -> code file and temp data file
 - Memory-mapped files -> memory-mapped files
 - When process ends, delete temp files
- Provides the illusion of an infinite amount of memory to programs

Zipf Model

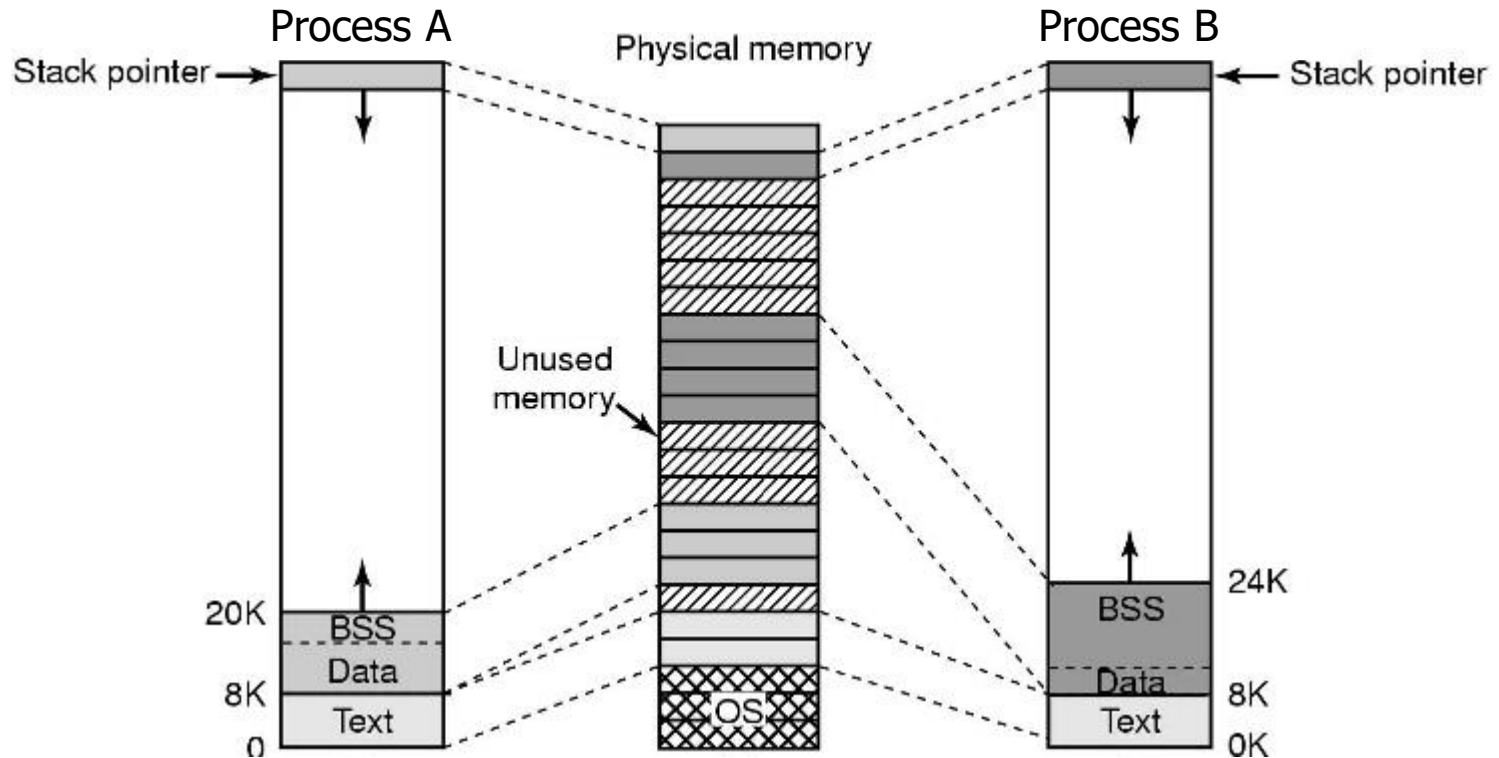
- Caching behavior of many systems are not well characterized by the working set model (e.g., web server)
- An alternative is the Zipf distribution model
 - Popularity $\sim \frac{1}{k^c}$ for k-th most popular item ($1 < c < 2$)
 - Examples: web pages, movies, words in text, etc..



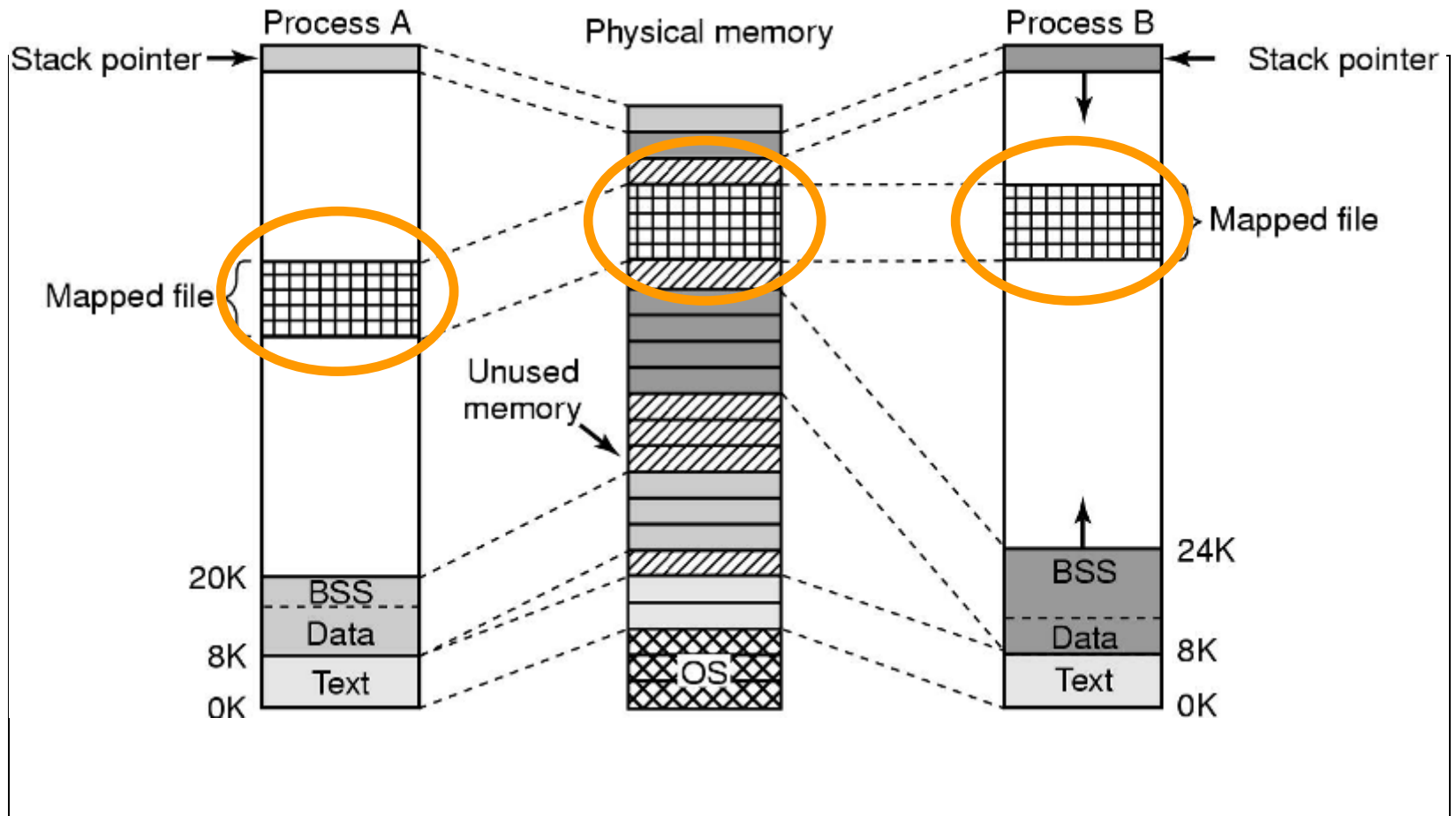
Memory management in Unix

- Since BSD v.3:
 - Paged segmentation
 - **Virtual memory** based on on-demand paging
- On demand paging:
 - ***Core map:***
 - kernel data structure that contains the allocation of the physical blocks
 - Used in case of page fault
 - **Page replacement algorithm:** Second Chance

Unix: memory organization



Unix: sharing memory mapped files



Models for Application File I/O

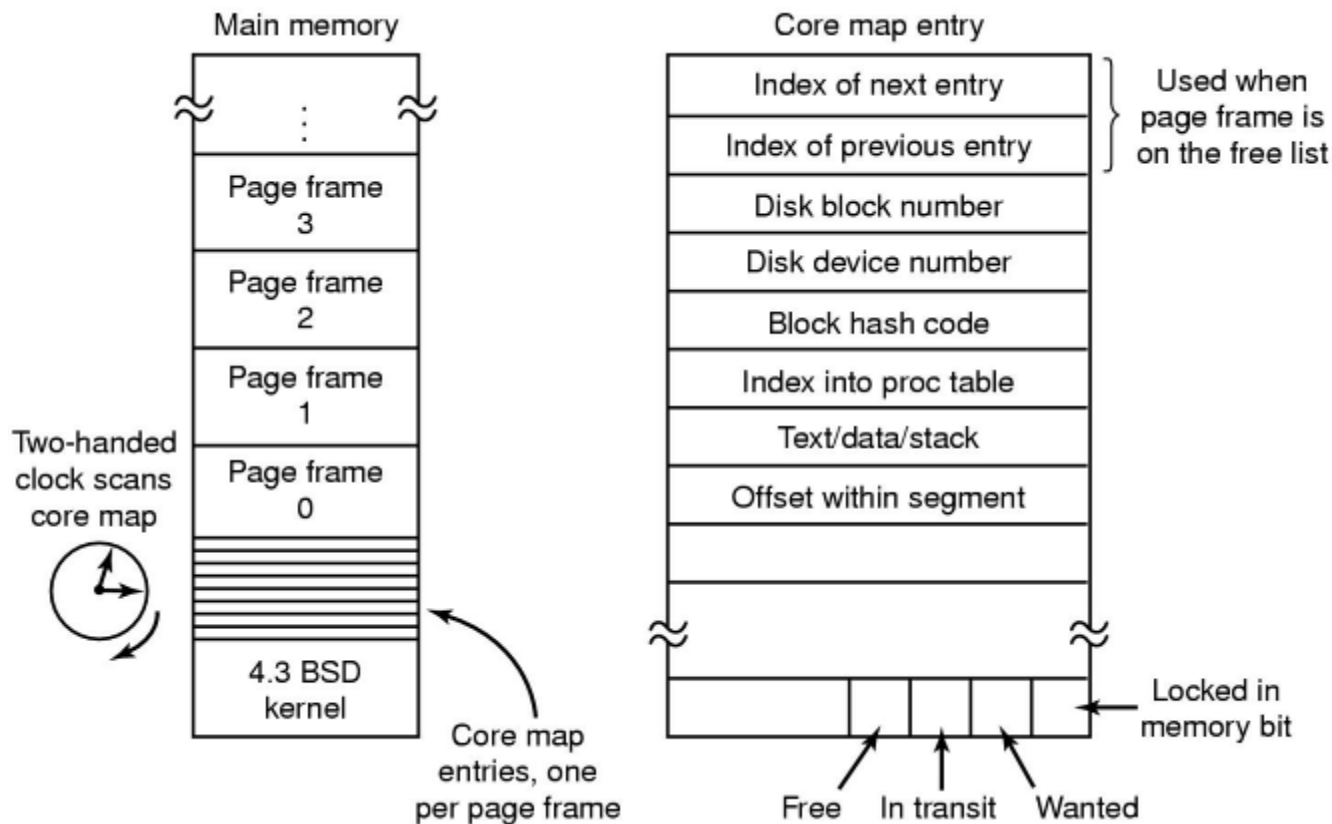
- Explicit read/write system calls
 - Data copied to user process using system call
 - Application operates on data
 - Data copied back to kernel using system call
- Memory-mapped files
 - Open file as a memory segment
 - Program uses load/store instructions on segment memory, implicitly operating on the file
 - Page fault if portion of file is not yet in memory
 - Kernel brings missing blocks into memory, restarts process

Advantages to Memory-mapped Files

- Programming simplicity, especially for large file
 - Operate directly on file, instead of copy in/copy out
- Zero-copy I/O
 - Data brought from disk directly into page frame
- Pipelining
 - Process can start working before all the pages are populated
- Interprocess communication
 - Shared memory segment vs. temporary file

Paging in Unix BSD

with an inverse page table (core map):



1) Core Map + Tabelle delle pagine

SO	SO	SO	SO	SO	SO		A,1 2	B,0 10	C,1 3		B,6 5	C,7 8		C,3 6	A,5 9	C,5 7	B,2 1	A,7 4	Processo, pagina Tempo ultimo riferimento
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	Blocco

Pagina	Blocco
0	-
1	7
2	-
3	-
4	-
5	15
6	-
7	18

Processo A

Pagina	Blocco
0	8
1	-
2	17
3	-
4	-
5	-
6	11
7	-

Processo B

Pagina	Blocco
0	-
1	9
2	-
3	14
4	-
5	16
6	-
7	12

Processo C

Tabelle delle pagine indicizzate da indice di pagina

- l'indice di pagina virtuale non è contenuto nella tabella

2) Core Map = Tabella delle pagine inversa

SO	SO	SO	SO	SO	SO		A,1 2	B,0 10	C,1 3		B,6 5	C,7 8		C,3 6	A,5 9	C,5 7	B,2 1	A,7 4	Processo, pagina Tempo ultimo riferimento
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	Blocco

Core Map indicizzate da indice di blocco

--> accesso con funzione hash

In entrambi i casi: vettore circolare dell'algoritmo *Second Chance* realizzato su *Core Map*, con i soli descrittori di blocchi assegnati ai processi

Page replacement in Unix (BSD) - I

Page replacement algorithm:

- Second chance (global)
- or variants (e.g. the two-handed clock algorithm)

Page replacement executed periodically by the *Page Daemon*:

- Uses parameters: *lotsfree*, *desfree*, *minfree*, with:

$$\textit{lotsfree} > \textit{desfree} > \textit{minfree}$$

Page replacement in Unix (BSD) - II

PageDaemon algorithm (**sketch**):

- *if* ($\#freeblocks \geq lotsfree$) return *//no operation required*
- *if* ($minfree \leq \#freeblocks < lotsfree$) **or**
 ($\#freeblocks < minfree$ **and**
 Average[$\#freeblocks, \Delta t$] $\geq desfree$))
 replace pages until $\#freeblocks = lotsfree + k$ (with $k > 0$)
- *if* (($\#freeblocks < minfree$ **and**
 Average[$\#freeblocks, \Delta t$] $< desfree$))
 swapout processes

Page replacement in Unix (BSD) - III

Relationship with the working set theory:

- If $\#freeblocks < minfree$:
 - High number of page faults from the last execution of *Page Daemon*
 - There exist processes that have $WS \not\subset RS$ (not included) that cause thrashing
 - *RS*: resident set, i.e. set of pages resident in memory
 - *WS*: working set
- If $average[\#freeblocks, \Delta t] < desfree$:
 - The problem has been there for a while
 - The swapout of some processes will free resources and avoid thrashing
 - The processes with $WS \not\subset RS$ will expand their resident set, as consequence of their future page faults

Swapping of processes in Unix

Swapout:

if (#freeblocks < minfree) and

(Average[#freeblocks, Δt] < desfree)

// The average is computed over a given time frame Δt

The *Page Daemon* selects «victim» processes based on :

- Priority
- Elapsed time without being executed
- Amount of memory required
- ...

Swaps out one or more victim process

until *#freeblocks* $\geq$ *lotsfree* + *k* (with $k > 0$)

Swapping of processes in Unix

Swapin:

If the number of free blocks is large enough, the *Page Daemon* selects one or more processes based on:

- Time spent in swapped-out state
- Amount of memory required
- ...

Swapin of one or more processes provided $\#freeblocks \geq lotsfree + k$ (with $k > 0$)

Windows: page fault management (1)

- Windows adopts a working set algorithm
 - Local algorithm
 - Operates on the set of *resident pages* (RS)
 - For a given process, $x=\#RS$ ranges in $[min,max]$
 - min and max are initially set to default values...
 - ... but they vary during the life of a process, to adapt to its memory needs

Windows: page fault management (2)

- At page fault of process P , the requested page is always loaded in a free block in memory
- Hence the size of the resident set of P increases ($x=x+1$)
- If $x \geq \textit{max}$ the pages of P in excess will be removed by the working set manager

Windows: page fault management (3)

Windows ensures there are always free blocks in memory by means of two processes:

- balance set manager
 - Executes periodically
 - If free memory is scarce: evicts pages identified by the working set manager
- working set manager
 - Implements the page replacement policy

Windows: working set manager

Adopts a working set page replacement policy

- For each process with $x > max$:
 - For each page p with *reference bit* $R = 1$: resets R and $count(p)$
 - For each page p with $R = 0$: increase $count(p)$
 - $count(p)$ is an approximation of the time of last reference (past distance) of a page
 - Set for removal $x - max$ pages in decreasing order of $count(p)$
- If the number of free blocks is still low: remove pages also of processes with $x > min$

Windows: management of pages

lists of pages and transitions

